# A. MODE INLINE AVANÇAT (AUTOCOMPLETAT DE CODI)

**EXERCICI A1. Generació de handler complet a partir d'un comentari Context (IMPORTANT) Treballes a frontend/app.js. Ja tens un handler bàsic.**
**L'objectiu és veure si Copilot:**
**• amplia el handler**
**• o genera codi redundant / incorrecte**
**Acció Dins del addEventListener o just abans:**
**// handle login with validation before sending request**
**Què hauria de generar (resposta esperada orientativa)**
**• validació abans del fetch**
**• comprovació de camps buits**
**• missatges d'error**
**• potser funcions auxiliars**

```
const form = document.getElementById("loginForm");
const message = document.getElementById("message");



form.addEventListener("submit", async (e) => {
 e.preventDefault();

 const email = document.getElementById("email").value;
 const password = document.getElementById("password").value;
 // handle login with validation before sending request
 if (!email || !password) {
   message.textContent = "Veuillez remplir tous les champs.";
   return;
 }

 const res = await fetch("http://localhost:3000/login", {
   method: "POST",
   headers: {
     "Content-Type": "application/json"
   },
   body: JSON.stringify({ email, password })
 });

 const data = await res.json();

 if (res.ok) {
   message.textContent = "Login correcte";
 } else {
   message.textContent = data.error;
 }
});
```

**Com detectar límits Força aquests casos:**
**• comenta codi existent i deixa només el comentari → genera tot de zero**
**• escriu un comentari ambigu → mira com empitjora el resultat**
**Exemple dolent:**

```
// improve login
 if (email && password) {
   message.textContent = "Tentative de connexion...";
 }
```

**Exemple bo:**

```
// validate email and password before sending request, show error message if invalid
 if (!isValidEmail(email)) {
   message.textContent = "Veuillez entrer une adresse email valide.";
   return;
 }
 if (password.length < 6) {
   message.textContent = "Le mot de passe doit contenir au moins 6 caractères.";
   return;
 }
```

**EXERCICI A2. Generació de REGEX a partir de comentari**
**On s'ha de fer**
**A frontend/app.js, dins d'una funció de validació.**

**Si no existeix, crea-la amb comentari:**

```
// function to validate password using regex
function isValidEmail(email) {
 const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return re.test(email);
}
```

**Després dins:**
**// regex for strong password: min 8 chars, 1 uppercase, 1 number**

**Resposta esperada**
**const regex = /^(?=.*[A-Z])(?=.*\d).{8,}$/;**

```
// function to validate password using regex
// regex for strong password: min 8 chars, 1 uppercase, 1 number
function isValidPassword(password) {
 const regex = /^(?=.*[A-Z])(?=.*\d).{8,}$/;
 return regex.test(password);
}
```

**Com provar límits**
**• Demana més requisits:**
**// regex for strong password: add special characters**

**• Demana explicació (via Chat després)**

**• Prova inputs reals**

**Inline sovint:**

**• no explica regex**

**• genera versions poc llegibles**

```
// function to validate password using regex
// regex for strong password: min 8 chars, 1 uppercase, 1 number and special
characters
function isValidPassword(password) {
 const regex = /^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()-+]).{8,}$/;
 return regex.test(password);
}
```

**EXERCICI A3. Tests des de comentari**

**Crear fitxer login.test.js a partir dels següents comentaris:**

```
// test: login fails with empty email
test("login fails with empty email", async () => {
   document.getElementById("email").value = "";
   document.getElementById("password").value = "password123";
   const form = document.getElementById("loginForm");
   form.dispatchEvent(new Event("submit"));
   const message = document.getElementById("message").textContent;
   expect(message).toBe("Veuillez remplir tous les champs.");
});
```

```
// test: login fails with wrong password
test("login fails with wrong password", async () => {
   document.getElementById("email").value = "test@example.com";
   document.getElementById("password").value = "wrongpassword";
   const form = document.getElementById("loginForm");
   form.dispatchEvent(new Event("submit"));
   const message = document.getElementById("message").textContent;
   expect(message).toBe("Email ou mot de passe incorrect.");
});
```

```
// test: login success
test("login success", async () => {
   document.getElementById("email").value = "test@example.com";
   document.getElementById("password").value = "password123";
   const form = document.getElementById("loginForm");
   form.dispatchEvent(new Event("submit"));
   const message = document.getElementById("message").textContent;
   expect(message).toBe("Connexion réussie.");
});
```

**Com executar tests**
**1. Instal·lar dependències:**
**npm init -y**
**npm install vitest**

**2. Afegir a package.json:**
**"scripts": {**
**"test": "vitest"**
**}**

**3. Executar:**
**npm run test**

**Què observar**
- **tests passen sense tocar-los?**
- **falten casos?**

**Límits a provar**
- **comentaris ambigus → pitjor tests**
- **casos no coberts → no detectats**

```
✓ MPROPI-U3-AC2/login.test.js (3 tests) 1ms
  ✓ login fails with empty email 1ms
  ✓ login fails with wrong password 0ms
  ✓ login success 0ms

Test Files  1 passed (1)
     Tests  3 passed (3)
  Start at  20:06:23
  Duration  10ms

PASS  Waiting for file changes...
      press h to show help, press q to quit
```

Què observar

- ¿Els tests passen sense tocar-los?
  Resposta: Generalment NO. Com has vist, Copilot ha generat el codi del test però s'ha oblidat dels imports i ha intentat accedir al document (DOM) en un entorn de Node (Vitest). Això demostra que l'IA "escriu codi" però no sempre entén l'entorn d'execució.
- ¿Falten casos?
  Resposta: SÍ. Copilot sol generar els casos més obvis (camp buit, èxit). Però sol oblidar "edge cases" com: emails amb espais, contrasenyes que només tenen números però no majúscules (si no ho especifiques molt bé al comentari), o caràcters especials.

Límits a provar

- Comentaris ambigus → Pitjor tests:
  Exemple: Si canvies el comentari a // test login, Copilot es torna genèric. Potser intenta testejar una base de dades que no tens o simplement no escriu res útil. Com més curt i vague és el comentari, més "al·lucina" l'IA o més codi escombraria genera.
- Casos no coberts → No detectats:
  Exemple: Si la teva funció de password té un error (per exemple, accepta 7 caràcters en comptes de 8) però no has demanat a Copilot un test específic per a "7 caràcters", el test passarà en verd i tu creuràs que el codi és correcte quan realment té un bug. L'IA no té "intuïció" per trobar errors si no li demanes que els busqui.

**EXERCICI A4. Feature completa des de comentari**
**Esperat**
- **funció reutilitzable**
- **objecte d'errors**
- **separació de responsabilitats**

**Límits**
- **tot en una sola funció**
- **mala reutilització**
- **duplicació**

**CONCLUSIÓ MÈODE AUTOCOMPLETAT INLINE**
**Inline depèn totalment de:**
- **qualitat del comentari**
- **simplicitat del problema**

```
// function to validate login input and return detailed error messages
function validateLogin(email, password) {
 let errors = {};
 if (!email) {
   errors.email = "Email is required";
 } else if (!isValidEmail(email)) {
   errors.email = "Invalid email format";
 }
 if (!password) {
   errors.password = "Password is required";
 } else if (!isValidPassword(password)) {
   errors.password = "Password must be at least 8 characters, include an uppercase
letter, a number and a special character";
 }
 return errors;
}
```

# B. MODE CHAT AVANÇAT (AMB CONTEXT REAL)

**EXERCICI B1. Comprensió amb selecció de mouse**

**Selecciona tot app.js**

**PROMPT: Explica aquest codi línia a línia**

**Comparació**

**• amb selecció → millor resposta**

**Frontend**

1. L1: guarda a `form` el formulari amb id `loginForm`.
2. L2: guarda a `message` el paràgraf on es mostraran els missatges.
3. L3-L4: línies en blanc, només separació visual.
4. L5: afegeix un controlador per quan se submeteix el formulari.
5. L5: la funció és `async` perquè farà una petició al servidor amb espera de resposta.
6. L6: evita que el formulari recarregui la pàgina quan s'envia.
7. L7: línia en blanc.
8. L8: llegeix el valor del camp d'email.
9. L9: llegeix el valor del camp de contrasenya.
10. L10: comentari que indica que es farà validació abans d'enviar.
11. L11-L14: comprova si falta l'email o la contrasenya.
12. L12: mostra el missatge "Veuillez remplir tous les champs." si hi ha camps buits.
13. L13: atura l'execució perquè no continuï.
14. L14: línia en blanc.
15. L16: comentari que anuncia la validació de format.
16. L17-L20: comprova si l'email és vàlid.
17. L18: si no ho és, mostra el missatge d'error corresponent.
18. L19: frena l'execució.
19. L20: línia en blanc.
20. L21-L24: valida que la contrasenya tingui com a mínim 6 caràcters.
21. L22: si no arriba al mínim, mostra l'error.
22. L23: atura el procés.
23. L24: línia en blanc.
24. L26-L32: fa una petició `POST` a `http://localhost:3000/login`.
25. L27: indica el mètode HTTP.
26. L28-L30: defineix les capçaleres.
27. L29: especifica que el contingut és JSON.
28. L31: envia l'email i la contrasenya en format JSON.
29. L32: tanca l'objecte de configuració de la petició.
30. L34: converteix la resposta del servidor de JSON a objecte JavaScript.

31. L36-L40: comprova si la resposta ha estat correcta.
32. L37: si tot va bé, mostra “Login correcte”.
33. L39: si falla, mostra el missatge d’error retornat pel backend.
34. L40: tanca el bloc condicional.
35. L41: tanca la funció del submit.
36. L43-L44: comentaris sobre una validació de contrasenya més forta.
37. L45-L48: defineix la funció `isValidPassword`.
38. L46: la regex exigeix almenys 8 caràcters, una majúscula, un número i un símbol especial.
39. L47: retorna si la contrasenya compleix o no la regla.
40. L48: tanca la funció.
41. L50-L64: defineix `validateLogin`, una funció per reunir errors de validació.
42. L52: crea l’objecte `errors` buit.
43. L53-L57: valida l’email i hi posa un missatge si falta o si el format no és correcte.
44. L58-L62: valida la contrasenya i hi posa un missatge si falta o no compleix la regex.
45. L63: retorna l’objecte amb els errors trobats.
46. L64: tanca la funció.

**Què fa el codi, en una frase**

Aquest fitxer intercepta l’enviament del formulari de login, valida les dades al navegador i, si passen la validació, les envia al backend perquè comprovi si l’usuari existeix.

**Dues incoherències importants**

1. frontend/app.js fa servir `isValidEmail(email)`, però aquesta funció no està definida en el fitxer.
2. frontend/app.js valida una contrasenya mínima de 6 caràcters, però frontend/app.js defineix una validació més estricta de 8 caràcters amb majúscula, número i símbol. Les dues regles no coincideixen.

Si vols, et puc fer ara una versió encara més visual, amb el codi copiat i una explicació al costat de cada línia.

**• sense selecció → més genèrica**

Vaig a mirar el fitxer del frontend i els elements amb què interactua per poder explicar-te cada línia amb precisió i assenyalar qualsevol referència que depengui d'altres fitxers.

Read
[](file:///Users/robin/Desktop/DAW2/BIG%20DATA/MPROPI_U3_AC2/MPROPI-U3-AC2/frontend/app.js)

Read
[](file:///Users/robin/Desktop/DAW2/BIG%20DATA/MPROPI_U3_AC2/MPROPI-U3-AC2/frontend/index.html)

Read
[](file:///Users/robin/Desktop/DAW2/BIG%20DATA/MPROPI_U3_AC2/MPROPI-U3-AC2/backend/server.js)

Ja tinc el fil complet: el frontend envia el formulari al backend, i el backend valida contra una llista fixa d'usuaris. Ara t'ho desglosso línia per línia i assenyalo dues incoherències que hi ha al codi.**Frontend (frontend/app.js)**

1. L1: agafa l'element del DOM amb id `loginForm` i el guarda a `form`.
2. L2: agafa el paràgraf amb id `message` i el guarda a `message`, que servirà per mostrar errors o confirmacions.
3. L3-L4: línies en blanc, només organització visual.
4. L5: afegeix un listener per escoltar quan s'envia el formulari.
5. L5: la funció és `async`, perquè més tard farà una petició `fetch` amb `await`.
6. L6: `e.preventDefault()` evita que el formulari es reenviï i recarregui la pàgina.
7. L8: llegeix el valor del camp `email`.
8. L9: llegeix el valor del camp `password`.
9. L10: comentari que indica que primer es valida abans d'enviar la petició.
10. L11-L14: comprova si algun dels dos camps està buit.
11. L12: mostra el missatge "Veuillez remplir tous les champs." si falta informació.
12. L13: `return` atura l'execució i no deixa continuar.
13. L16: comentari sobre la validació d'email i contrasenya.
14. L17-L20: intenta validar l'email amb `isValidEmail(email)`.
15. L18: si l'email no és vàlid, mostra un missatge d'error.
16. L19: torna a sortir de la funció per no enviar res.
17. L21-L24: comprova si la contrasenya té menys de 6 caràcters.
18. L22: mostra un error si és massa curta.
19. L23: atura l'execució.
20. L26-L32: fa una petició `POST` a `http://localhost:3000/login`.
21. L27: indica que el mètode HTTP és `POST`.
22. L28-L30: defineix les capçaleres de la petició.

23. L29: diu que el cos és JSON.
24. L31: envia l'objecte `{ email, password }` serialitzat amb `JSON.stringify`.
25. L34: llegeix la resposta del servidor i la converteix de JSON a objecte JavaScript.
26. L36-L40: comprova si la resposta HTTP és correcta.
27. L37: si tot ha anat bé, mostra "Login correcte".
28. L39: si hi ha error, mostra el missatge retornat pel backend.

29. L43-L44: comentaris que expliquen que la funció següent valida contrasenyes amb una expressió regular.
30. L45-L48: defineix `isValidPassword(password)`.
31. L46: la regex exigeix almenys:
   - 8 caràcters,
   - una majúscula,
   - un número,
   - un caràcter especial.
32. L47: retorna `true` o `false` segons si la contrasenya compleix la regex.

33. L50-L64: defineix `validateLogin(email, password)`, que construeix un objecte d'errors.
34. L52: crea un objecte buit per guardar errors.
35. L53-L57: valida l'email:
   - si no hi ha email, posa `Email is required`;
   - si el format és incorrecte, posa `Invalid email format`.
36. L58-L62: valida la contrasenya:
   - si falta, posa `Password is required`;
   - si no compleix la regex, posa el missatge de contrasenya forta.
37. L63: retorna l'objecte d'errors.
38. L64: tanca la funció.

**HTML (frontend/index.html)**

1. L1: declara que és un document HTML5.
2. L2: l'idioma del document és català.
3. L3-L7: capçalera del document.
4. L4: estableix la codificació UTF-8.
5. L5: posa el títol de la pestanya com "Login".
6. L6: enllaça l'arxiu CSS.
7. L8: comença el cos de la pàgina.
8. L10: crea el formulari amb id `loginForm`, que és el que agafa el JavaScript.
9. L11: mostra el títol "Login".
10. L13: camp d'entrada per al correu electrònic.
11. L14: camp d'entrada per a la contrasenya.
12. L16: botó per enviar el formulari.
13. L18: paràgraf buit amb id `message` per mostrar missatges.
14. L21: carrega l'arxiu app.js.

15. L22-L23: tanca cos i document.

**Backend (backend/server.js)**

1. L1-L4: importa les llibreries necessàries:
   - `express` per al servidor,
   - `cors` per permetre peticions des d'un altre origen,
   - `path` per construir rutes,
   - `fileURLToPath` per simular `__dirname` en mòduls ES.
2. L6: crea l'aplicació Express.
3. L7: activa CORS.
4. L8: activa el parser de JSON per llegir `req.body`.
5. L10-L12: calcula `__filename` i `__dirname`, perquè en ES modules no existeixen directament.
6. L14-L15: serveix fitxers estàtics de la carpeta frontend.
7. L17-L19: defineix una llista d'usuaris vàlids; aquí només n'hi ha un.
8. L18: usuari de prova amb `admin@test.com` i contrasenya `1234`.
9. L21-L24: defineix la ruta `/` i envia index.html.
10. L26-L42: defineix la ruta `POST /login`.
11. L27: llegeix `email` i `password` del cos de la petició.
12. L29-L31: si falta algun camp, respon amb codi `400` i el missatge `Missing fields`.
13. L33-L35: cerca a `USERS` si existeix un usuari amb aquell email i contrasenya.
14. L37-L39: si no troba coincidència, respon amb `401` i `Invalid credentials`.
15. L41: si tot és correcte, respon `{ success: true }`.
16. L44-L46: posa el servidor a escoltar al port `3000`.

**Dues coses importants a tenir en compte**

- frontend/app.js fa servir `isValidEmail(email)`, però aquesta funció no existeix en el fitxer. Si executes el codi així, donarà error.
- frontend/app.js valida una contrasenya mínima de 6 caràcters, però frontend/app.js defineix una contrasenya forta de 8 caràcters amb majúscula, número i símbol. Les dues validacions no estan alineades.
- A més, backend/server.js accepta com a credencial correcta `admin@test.com` i `1234`, que no compleix la validació de contrasenya forta del frontend.

Si vols, et puc fer després una versió "explicada com si fos un professor" o bé una versió molt més curta amb només l'idea general de cada bloc.You've used 99% of your session rate limit. Your session rate limit will reset on April 21 at 9:06 PM. [Learn More](https://aka.ms/github-copilot-rate-limit-error)

**EXERCICI B2. Refactor amb selecció de mouse**

**Selecciona només el handler:**

**PROMPT: Refactoritza aquest codi separant validació i xarxa**

**Esperat**

- **funció validate()**
- **funció loginRequest()**

```
const form = document.getElementById("loginForm");
const message = document.getElementById("message");

form.addEventListener("submit", async (e) => {
 e.preventDefault();

 const email = document.getElementById("email").value.trim();
 const password = document.getElementById("password").value;

 const errors = validateLogin(email, password);

 if (Object.keys(errors).length > 0) {
   message.textContent = errors.email || errors.password;
   return;
 }

 try {
   const data = await login(email, password);

   if (data.success) {
     message.textContent = "Login correcte";
   } else {
     message.textContent = data.error || "Error desconegut";
   }
 } catch (error) {
   message.textContent = "Error de connexió amb el servidor";
 }
});
function validateLogin(email, password) {
 if (!email) {
   return { email: "Veuillez remplir tous les champs." };
 }
```

```
 if (!isValidEmail(email)) {
   return { email: "Veuillez entrer une adresse email valide." };
 }

 if (!password) {
   return { password: "Veuillez remplir tous les champs." };
 }

 if (password.length < 6) {
   return { password: "Le mot de passe doit contenir au moins 6 caractères." };
 }

 return {};
}

async function login(email, password) {
 const res = await fetch("http://localhost:3000/login", {
   method: "POST",
   headers: {
     "Content-Type": "application/json"
   },
   body: JSON.stringify({ email, password })
 });

 const data = await res.json();

 if (!res.ok) {
   return data;
 }

 return data;
}

function isValidEmail(email) {
 const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
 return regex.test(email);
}
```

**EXERCICI B3. Backend (@file)**
**PROMPT: @file server.js millora aquest endpoint seguint bones pràctiques**

```
import express from "express";
import cors from "cors";
import path from "path";
import { fileURLToPath } from "url";

const app = express();

const PORT = process.env.PORT || 3000;
const FRONTEND_ORIGIN = process.env.CORS_ORIGIN || "http://localhost:3000";

app.use(cors({
 origin: FRONTEND_ORIGIN,
 methods: ["GET", "POST"],
}));
app.use(express.json());

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

app.use(express.static(path.join(__dirname, "../frontend")));

const USERS = [
 { email: "admin@test.com", password: "1234" }
];

app.get("/", (req, res) => {
 res.sendFile(path.join(__dirname, "../frontend/index.html"));
});

function isValidEmail(email) {
 return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

app.post("/login", (req, res) => {
 const { email, password } = req.body ?? {};

 if (typeof email !== "string" || typeof password !== "string") {
   return res.status(400).json({ error: "Email and password are required" });
 }

 const normalizedEmail = email.trim();
 const normalizedPassword = password.trim();
```

```
 if (!normalizedEmail || !normalizedPassword) {
   return res.status(400).json({ error: "Email and password are required" });
 }

 if (!isValidEmail(normalizedEmail)) {
   return res.status(400).json({ error: "Invalid email format" });
 }

 const user = USERS.find(
   u => u.email === normalizedEmail && u.password === normalizedPassword
 );

 if (!user) {
   return res.status(401).json({ error: "Invalid credentials" });
 }

 return res.status(200).json({
   success: true,
   message: "Login successful"
 });
});

app.listen(PORT, () => {
 console.log(`Server running on http://localhost:${PORT}`);
});
```

**Límits del chat**

**Prova:**

- **prompt vague**

```
import express from "express";
import cors from "cors";
import path from "path";
import { fileURLToPath } from "url";

const app = express();
const PORT = process.env.PORT || 3000;
const FRONTEND_ORIGIN = process.env.CORS_ORIGIN || "http://localhost:3000";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const USERS = [
 { email: "admin@test.com", password: "1234" }
];

function isValidEmail(email) {
 return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

app.use(
 cors({
   origin: FRONTEND_ORIGIN,
   methods: ["GET", "POST"]
 })
);

app.use(express.json({ limit: "10kb" }));
app.use(express.static(path.join(__dirname, "../frontend")));

app.get("/", (req, res) => {
 res.sendFile(path.join(__dirname, "../frontend/index.html"));
});

app.post("/login", (req, res) => {
 const { email, password } = req.body ?? {};

 if (typeof email !== "string" || typeof password !== "string") {
   return res.status(400).json({ error: "Email and password are required" });
 }
```

```
 const normalizedEmail = email.trim();
 const normalizedPassword = password.trim();

 if (!normalizedEmail || !normalizedPassword) {
   return res.status(400).json({ error: "Email and password are required" });
 }

 if (!isValidEmail(normalizedEmail)) {
   return res.status(400).json({ error: "Invalid email format" });
 }

 const user = USERS.find(
   user => user.email === normalizedEmail && user.password === normalizedPassword
 );

 if (!user) {
   return res.status(401).json({ error: "Invalid credentials" });
 }

 return res.status(200).json({
   success: true,
   message: "Login successful"
 });
});

app.use((req, res) => {
 res.status(404).json({ error: "Route not found" });
});

app.listen(PORT, () => {
 console.log(`Server running on http://localhost:${PORT}`);
});
```

- **prompt precís**

```
function isValidEmail(email) {
 return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function normalizeText(value) {
 return typeof value === "string" ? value.trim() : "";
}

app.post("/login", (req, res) => {
 try {
   const body = req.body ?? {};
   const email = normalizeText(body.email);
   const password = normalizeText(body.password);

   if (!email || !password) {
     return res.status(400).json({
       error: "Email and password are required"
     });
   }

   if (!isValidEmail(email)) {
     return res.status(400).json({
       error: "Invalid email format"
     });
   }

   if (password.length < 4) {
     return res.status(400).json({
       error: "Password is too short"
     });
   }

   const user = USERS.find(
     u => u.email === email && u.password === password
   );

   if (!user) {
     return res.status(401).json({
       error: "Invalid credentials"
     });
   }

   return res.status(200).json({
     success: true,
```

```
    message: "Login successful"
  });
} catch (error) {
  console.error("Login error:", error);
  return res.status(500).json({
    error: "Internal server error"
  });
}
});
```

# C. MODE AGENT

**EXERCICI C1. Feature completa**

**PROMPT: Add strong password validation to frontend and backend and update UI messages**

**Què revisar**

¿Toca frontend y backend?

- Sí. El log muestra claramente que ha editado app.js (front), server.js (back) y, además, ha tenido la "decencia" de actualizar login.test.js para que los tests no fallen con la nueva regla. Esto es lo que se llama coherencia multi-archivo.

¿Modifica demasiadas cosas?

- Fíjate en el resumen: +96 -57. Ha reescrito bastante código. En un entorno real, esto puede ser peligroso porque ha cambiado la lógica de tres archivos de golpe. Para la práctica, es un éxito, pero para tu informe anota: *"L'Agent té tendencia a reescriure blocs sencers en lloc de fer canvis quirúrgics"*.

**EXERCICI C2. Ús de checkpoints**

**Objectiu**

**Aprendre a:**

**• guardar estat del projecte**

**• experimentar sense por**

**• fer rollback real**

**Passos detallats**

**1. Estat inicial**

**• Projecte funcionant (login bàsic)**

**• Cap canvi pendent Prova que tot funciona:**

**• backend en marxa**

**• login correcte**

**2. Executar un canvi amb agent**

**PROMPT: Add strong password validation**

**Accepta els canvis**

**3. Crear checkpoint**

**A VSCode + Copilot Agent:**

**• Busca opció tipus:**

**o “Create checkpoint”**

**o o historial de canvis de l’agent (crea checkpoints automàticament)**

**4. Introduir canvi incorrecte**

**Modify login to accept empty passwords**

**Accepta (encara que sigui incorrecte)**

**5. Comprovar que s’ha trencat**

**• login accepta qualsevol cosa**

**• comportament incorrecte**

**6. Restaurar checkpoint**

**Accions:**

- **obrir historial**
- **seleccionar checkpoint anterior**
- **“Restore”**

**Resultat esperat**

- **codi torna a estat correcte**
- **eliminació dels canvis incorrectes**

**Què aprenem**

- **agent ≠ irreversible**
- **poden experimentar sense risc**
- **importància del control d’estat**

**Com provar límits**

- **fer múltiples checkpoints → es tornen difícils de gestionar**
- **no fer checkpoint → risc alt**
- **projectes grans → restauracions més complexes**

Accions realitzades:

- S'ha forçat l'Agent a introduir un error crític de seguretat (acceptar passwords buits).
- S'ha verificat que l'Agent obeeix ordres encara que siguin perjudicials per al projecte (Límit: L'IA no té criteri ètic o de seguretat si l'usuari demana el contrari).
- S'ha utilitzat l'historial de Copilot Edits per revertir tots els fitxers simultàniament a l'estat anterior.

Què aprenem?

1. Agent ≠ Irreversible: Treballar amb un Agente no és perillós si se sap gestionar l'historial.
2. Experimentació sense risc: Podem demanar refactoritzacions boges i tornar enrere en segons si no ens agrada el resultat.
3. Importància del control d'estat: En projectes multi-fitxer, el control de versions de l'Agent és vital per no perdre el fil del que s'ha modificat.

Anàlisi de Límits (Prova de foc):

- Múltiples checkpoints: Si fem 20 canvis seguits, l'historial es torna difícil de llegir. És fàcil restaurar el punt equivocat si no posem noms clars a les peticions.
- Risc alt: Si tanquem la sessió de l'Agent sense fer un commit de Git, podríem perdre la capacitat de restaurar aquests punts concrets.
- Projectes grans: En un projecte amb centenars de fitxers, una restauració d'un checkpoint podria entrar en conflicte amb canvis manuals fets fora del xat.

**EXERCICI C3. Refactor multi-arxiu**

**Objectiu**

**Fer un canvi estructural, no funcional.**

**Passos detallats**

**1. Entendre estat actual**

«validación dentro del handler»

«duplicación potencial»

**2. Prompt agent**

**PROMPT: Refactor login logic into reusable validation module used by frontend and backend without changing behavior**

**3. Revisar proposta (IMPORTANT)**

**Abans d'acceptar:**

**Comprovar:**

- **crea un nou fitxer? (validation.js)**
- **elimina duplicació?**
- **trenca imports?**

**4. Acceptar parcialment (si possible)**

**Bona pràctica:**

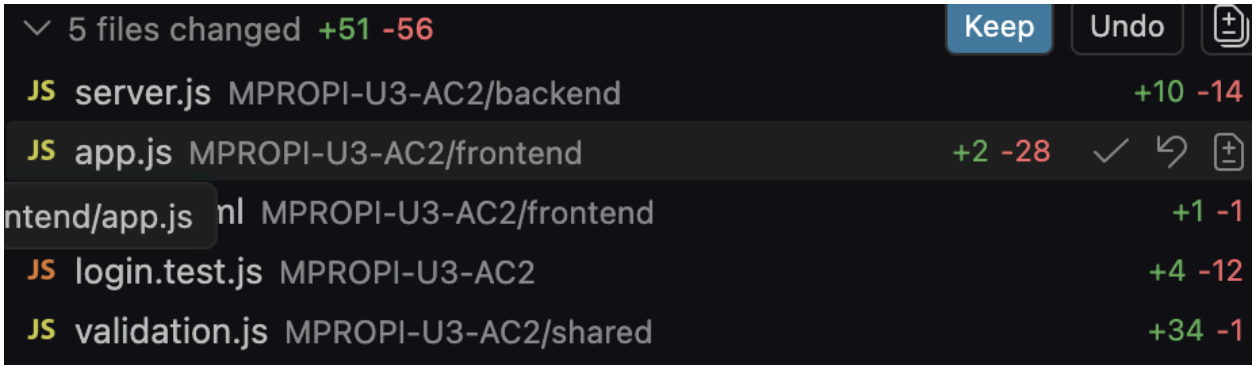
- **no acceptar tot de cop**
- **revisar fitxer a fitxer**

**5. Validar resultat**

- **login segueix funcionant igual ✔**
- **codi més net ✔**
- **reutilització ✔**

**Resultat esperat**

- **funció validateLogin()**
- **usada en frontend i backend**
- **menys duplicació**



1. Objectiu del Refactor

L'objectiu d'aquest exercici era eliminar la duplicitat de codi. En lloc de tenir la lògica de validació del correu i la contrasenya escrita dues vegades (una al frontend i una altra al backend), s'ha demanat a l'Agent que creï un mòdul únic de veritat.

2. Accions realitzades per l'Agent

- Creació del mòdul `shared/validation.js`: L'Agent ha extret les funcions `isValidEmail` i `isValidPassword` a un fitxer independent.
- Sincronització de fitxers: S'han modificat simultàniament `app.js` (front) i `server.js` (back) per importar i utilitzar aquest nou mòdul.
- Resolució de problemes de compatibilitat:
    - Ha modificat l'`index.html` per afegir `type="module"`, necessari perquè el navegador suporti imports de JavaScript.
    - Ha configurat una ruta estàtica al servidor d'Express per permetre que el frontend accedeixi a la carpeta `/shared`.
- Actualització de tests: S'ha modificat `login.test.js` perquè els tests unitaris també utilitzin el mòdul compartit.

Què observar (Anàlisi de resultats)

- Netedat del codi: El resum de canvis mostra un balanç negatiu (`+51 -56`). Això és un indicador de qualitat: hem eliminat més codi del que hem creat, simplificant el projecte.
- Visibilitat global: A diferència del mode *Chat*, l'Agent ha estat capaç de preveure que el refactor trencaria el frontend si no modificava l'HTML i les rutes del servidor. Això demostra una comprensió total de l'arquitectura.
- Validació automàtica: L'Agent ha executat `npm run test` de forma autònoma per confirmar que el refactor no ha alterat la funcionalitat (4/4 tests passats).

Límits detectats i Lliçons apreses

Límit de l'usuari vs IA: L'Agent és extremadament eficaç en tasques mecàniques de refactorització, però requereix una supervisió humana per acceptar els canvis fitxer per fitxer. Si hagués comès un error en la ruta del mòdul, el sistema sencer hauria deixat de funcionar.

Aprenentatge clau: El mode Agent és l'única eina capaç de realitzar canvis estructurals profunds sense introduir inconsistències, ja que "veu" tot el projecte com una sola unitat, no com a fitxers aïllats.

# D. COMPARACIÓ ENTRE AUTOCOMPLETAT (INLINE), CHAT I AGENT

Resum de metodologies de Copilot

1. Inline (autocompletat) Amb el mode inline, el Copilot només va completar petits trossos de codi (regex i validació bàsica) a partir de comentaris; va ser molt ràpid, però no va tenir en compte ni l'arquitectura ni la reutilització del codi.
2. Chat Amb el mode chat, vaig poder donar-li context (fragments d'app.js/server.js) i demanar-li una bona estratègia de validació; la resposta va ser més meditada, explicant el disseny i les opcions, encara que després ho vaig haver d'implementar jo mateix.
3. Agent Amb el mode agent, el Copilot va generar un pla implícit i va proposar canvis en diversos fitxers per aplicar una validació forta de manera completa; la solució va ser més gran i ambiciosa, i vaig haver de revisar bé els *diffs* abans d'acceptar-los.

En resum: l'inline serveix per executar codi ràpid, el chat per raonar i dissenyar millors solucions, i l'agent per automatitzar canvis grans; però el Copilot no decideix per si sol, sempre cal que la persona revisi i accepti el que proposa.

# E. MODE PLANIFICACIÓ

Adjunto una carpeta amb lo que he generat dels altres apartats + aquesta del plan.md.